

Part C Project Report

M Ahsan Al Mahir

§1 Project Overview

Most modern real-time operating systems (RTOS) for small microcontrollers allow a limited number of processes to run simultaneously, typically defined at compile time. However, this restricts the flexibility to create processes dynamically. In contrast, one of the earliest examples of an RTOS, deployed on the Apollo Guidance Computer (AGC), allowed latency-insensitive jobs to be declared at runtime while processes with stricter timing constraints were defined at compile time. The queue of processes scheduled to run at predefined intervals was called the **WAITLIST**. Longer **EXECUTIVE** jobs could be created dynamically with varying priorities, often initiated by **WAITLIST** jobs, and were executed using the remaining available processor time.

This flexible design made the AGC highly fault-tolerant. Famously, during the Apollo 11 landing, the AGC in the lunar module ran out of memory for new **EXECUTIVE** jobs, triggering a 1202 alarm. While low-priority tasks, such as updating the astronauts' displays, failed to execute, higher-priority jobs that controlled the spacecraft continued unaffected, allowing the crew to land safely.

The goal of this project is to implement a real-time operating system that models this approach on a small microcontroller and to explore the potential advantages of this design over the commonly used round-robin schedulers in modern RTOS implementations.

§2 Progress

The initial phase of the project involves implementing the AGC's scheduling algorithm on a modern microcontroller. Based on the advice of my supervisor, Prof. Alex Rogers, I selected the **micro:bit v1** device as the target platform. This device features an ARM Cortex-M0 processor and includes peripherals such as system timers, LEDs, and a radio module.



Figure 2.1: A **micro:bit v1** device

To begin, I familiarized myself with programming microcontrollers at the bare-metal level. I studied the book “Bare-metal Programming for ARM” [3], implementing its examples to gain hands-on experience. This process helped me quickly get up to speed with microcontroller programming.

Next, I set up a development framework for working with the physical `micro:bit` device. The book “Bare Metal `micro:bit`” by Mike Spivey [2] was particularly helpful, as it included hardware headers and reference manuals specific to the `micro:bit`, enabling a quick and efficient setup. I also developed automation tools to compile and deploy code to the `micro:bit` device directly from my laptop.

After establishing the development environment, I began investigating the AGC scheduler in detail. To deepen my understanding, I am studying the book “The Apollo Guidance Computer: Architecture and Operation” by Frank O’Brien [1]. Currently, I have a working implementation of the `WAITLIST` tasks, which employ a primitive form of multiprocessing using timer interrupts. The next step is to implement preemptive context switching and runtime process creation. Subsequently, I plan to complete the implementation of `EXECUTIVE` jobs from the AGC. To efficiently support priority-based multitasking with `EXECUTIVE` jobs, I intend to study algorithms used by modern operating systems like Linux and implement a simplified version tailored to the project.

To demonstrate the effectiveness of this scheduler, I plan to develop a minimal shell for the operating system. This shell will enable basic scripting capabilities and runtime process creation.

§3 Feedback

Progress on my project has been slower than anticipated this term due to my commitments to job hunting and PhD applications. However, I aim to accelerate during the Christmas break. So far, I have had two meetings with Prof. Alex Rogers to discuss the project’s progress and direction. His feedback has been constructive and helpful, providing valuable insights to guide my work.

References

- [1] Frank O’Brien. *The Apollo Guidance Computer: Architecture and Operation*. Jan. 2010. ISBN: 978-1-4419-0876-6. DOI: 10.1007/978-1-4419-0877-3.
- [2] Mike Spivey. *Bare metal micro:bit*. https://spivey.oriel.ox.ac.uk/baremetal/Bare_metal_micro:bit. 2019.
- [3] Daniels Umanovskis. *Bare-metal programming for ARM*. <https://github.com/umanovskis/baremetal-arm>. 2019.